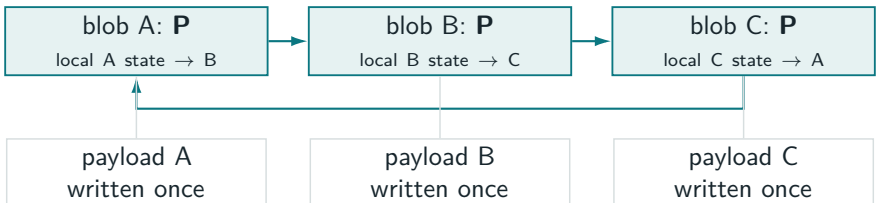
Atomic append-only storage, without writing payload twice

Participant-linked UNO / V2 and the contiguous journal simplification

Status: opt-in ALPHA API; ordinary Storage and V0/V1 remain unchanged.



The contract we actually need

last successfully synchronized epoch

↓

append bytes are visible to this handle

before publication, restart exposes **old**

successful `sync`: complete new epoch

↓

one blob or an all-or-nothing group

after success, restart exposes **new**

FAULT MODEL

Any subset of writes not covered by a completed durability barrier may survive.

INDETERMINATE WINDOW

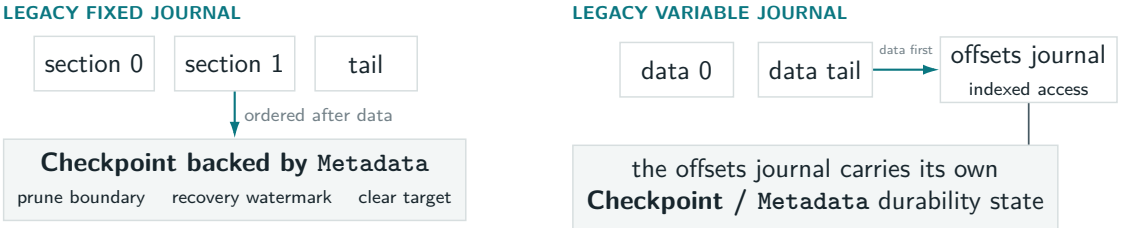
A failed or cancelled sync may recover old or complete new—never a mixture.

PERFORMANCE TARGET

Keep ordinary append behavior: write payload once and pay at the existing sync.

Atomicity is a crash-recovery property; durability remains attached to `sync`.

Before: each journal coordinated its own crash state



- `commit`, `start_sync`, and stronger `sync` kept data, indexes, and recovery hints in a safe order.
- Main had no filesystem primitive for atomically publishing several blob roots together.

The journals were correct, but every compound structure owned a bespoke durability protocol.

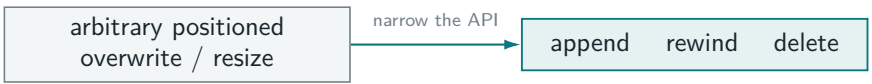
Why generic atomic writes were too expensive

REJECTED DIRECTIONS EXPLORED IN THIS PR—NOT BEHAVIOR SHIPPED ON MAIN

Shadow / redo payload
write new bytes aside, then copy or replay them
≈ 2× payload I/O

Filesystem COW
excellent with reflinks, but not a portable assumption
ext4 cannot be the slow path

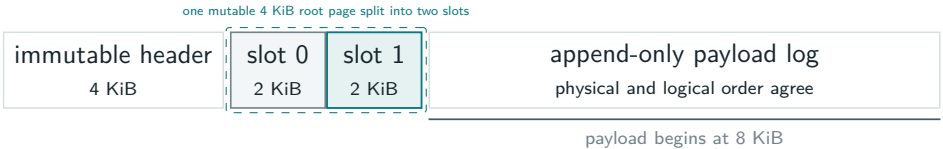
Coordinator file
another manifest, write, barrier, and recovery dependency
extra serialized metadata



A checksum can detect a torn overwrite; it cannot restore overwritten bytes. Append-only preserves the old prefix, so a root can select a complete visible prefix.

The decisive optimization is an API constraint: preserve old bytes and publish a new length.

V2: one root page, payload at final offsets



92-byte root header

spelling generation logical length

64-byte app tag CRC32C

One 2 KiB slot

92 B root + 16 B witness envelope

+ ≤1,940 B local link / unused space

APPEND

Write through immediately at the byte's final file offset.

PUBLISH

Alternate root slots; generation parity preserves the immediate fallback.

RECOVER

Read bounded roots and, only for an unresolved epoch, checksum a bounded suffix.

No extent map, hole punching, or compaction: a root selects a reachable payload prefix.

One blob: the durable decision fits in one barrier

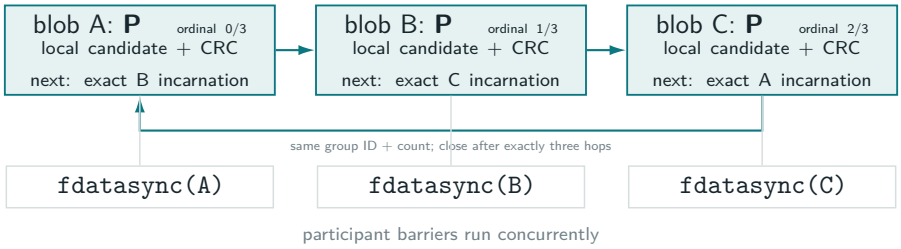
the old root is never overwritten
while publishing its successor

ON RESTART

- If the prepared root, self link, and required suffix CRC all validate: expose new.
- Otherwise: expose the preceding committed root.
- No follow-up P→C marker and no second durability barrier.

The self-link is the durable decision; later compatible epochs alternate and supersede it.

Several blobs: each root stores one link in a closed ring



WHAT ONE LOCAL WITNESS CARRIES

Fresh 128-bit group ID, count + ordinal, local incarnation/candidate/removal bit, bounded suffix CRC32C, and only the successor's partition/name/incarnation.

WHAT RESTART PROVES

Opening one unresolved P follows exactly the declared ring, growing state only after each valid link. Only one unique, consecutive, closed ring can publish the complete new group.

Live process: one typed `Decision` in RAM. On disk: distinct local links only.

No coordinator file and no replicated group descriptor: the participant files are the recovery index.

Overlapping atomic groups

EVERY RING SORTS PARTICIPANT KEYS: `(PARTITION, RAW BLOB NAME)`

caller: $\{B, A\}$ or $\{A, B\}$ → $A_0 \rightarrow B_1 \rightarrow A_0$ → same ordinals, links, and lock order

EXACT PARTICIPANT SET REPEATS

G_1 $A_0 \rightarrow B_1 \rightarrow A_0$ → G_2 $A_0 \rightarrow B_1 \rightarrow A_0$

Adjacent generations supersede directly; no P→M rewrite.

Protocol policy

disjoint rings need no common order.

PARTICIPANT SET CHANGES

G_1 $A \rightarrow B$ → wait for B materialize G_1 → G_2 $B \rightarrow C$

Shared-B serialization is required; eager materialization is current policy.

Prototype policy

process-local namespace lock

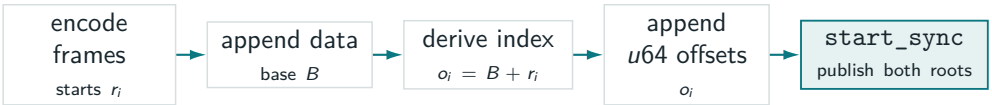
\+ singleton carried decision.

Lexicographic links remove caller-order ambiguity and deadlock cycles; they do not create a global commit order.

Dependent writes can be built, then published together

VARIABLE JOURNAL APPEND: SEQUENTIAL STAGING IS ALLOWED

build phase: pending writes may depend on earlier results

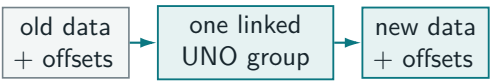


HOW VALUE / IS LOCATED

```
start = offsets[i]
end = offsets[i + 1]
decode data[start..end]
```

Active tail ends at committed data length;
sealed sections store that length as a sentinel.

WHAT BECOMES VISIBLE



Restart sees the old pair or complete new pair—never mismatched data and offsets lengths.

Stage dependency-aware writes first; one linked UNO group publishes the finished set.

Batch scale: fixed local metadata per participant

EVERY DIRTY PARTICIPANT: ONE 2 KIB SLOT CONTAINS ITS ROOT AND LOCAL WITNESS

92 B
candidate root

16 B
witness header

≤ 1,940 B local link
296 B fixed fields + one successor partition/name

PARTICIPANT METADATA

one link
candidate + successor

1,644 B
maximum successor path

Each participant names only its next exact incarnation. The paths are distributed once around the ring rather than gathered into one record.

PAYLOAD SIZE: NO PROTOCOL BATCH CAP

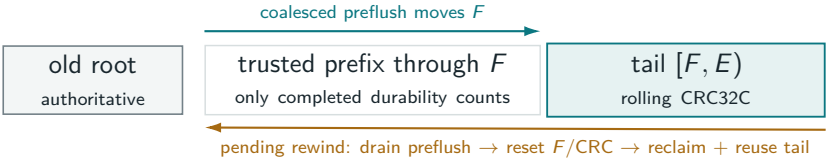
arbitrarily large epoch
streamed once + incrementally preflushed

At most **64 MiB aggregate** remains for restart CRC work. Older immutable bytes preflush while appends continue.

Metadata grows one local link per participant; payload is streamed once rather than copied.

Large epochs advance a trusted payload prefix

WHILE APPENDING: PAYLOAD IS AT FINAL OFFSETS; NO CANDIDATE ROOT EXISTS YET



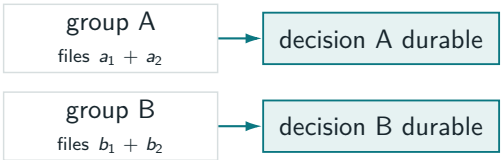
AT SYNC: FREEZE ONE EPOCH, THEN PUBLISH

CRASH OUTCOME

Repeat per participant: no payload-byte cap, at most 64 MiB aggregate restart tail, and concurrent final barriers.

Intentional limitation: no global sync order

INDEPENDENT GROUPS



No global order

no root WAL, durable global coordinator, LSN, or commit sequence

What the protocol avoids

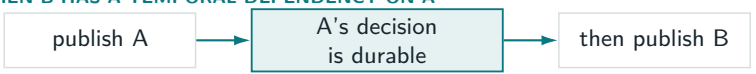
no required persistent global lock

or volume-like filesystem layer

CRASH STATES FOR INDEPENDENTLY STARTED A AND B

old A / old B | new A / old B | **old A / new B is possible** | new A / new B

WHEN B HAS A TEMPORAL DEPENDENCY ON A



old/old, new/old, new/new
never old A / new B

Atomic membership is local. Put a temporal dependency in one group, wait at its edge, or add a higher-level WAL only when a total order is required.

Adoption example: not every file needs V2

FUTURE FREEZER / IMMUTABLE ARCHIVE CONSUMER PATTERN—NOT PART OF THIS PR

KEEP ORDINARY IN-PLACE MATERIALIZATIONS

Freezer table	Ordinal
epoch slots + CRC	sparse slots + CRC

Old checkpoint hides epoch E slots and unset bits.

ADOPT V2 WHERE THE SHAPE IS APPEND-ONLY

value journal (append-only payload) → key-index journal (returned locations)

One UNO group prevents a mismatched value/index tail.

LOGICAL COMMIT WITH ONE EXPLICIT TEMPORAL DEPENDENCY

1. stage — pending epoch E
2. sync table + Ordinal — await; old checkpoint wins
3. one linked UNO group publishes journals + state E — application authority becomes visible

before UNO: **old** | in flight: **old or complete new** | after `Ok`: **new**

If old wins, future materializations remain unreachable; if new wins, their durability was already proven.

Ordinary indexes first; atomic authority second.

Recovery is correct for arbitrary surviving write subsets

What validates after restart	What recovery can prove	Exposed result
No M/T proof; every exact P, local link, incarnation, candidate, and suffix validates	One group ID/count; unique consecutive ordinals close after exactly *N* hops	New roots for the whole group
No M/T proof; any link, incarnation, candidate, or suffix is missing/mismatched	The linked group decision is incomplete	Old roots for the whole group
An exact materialized M or tombstone T validates	Installation began only after the P ring was durable	Finish peers; after unlink begins, resume the descending frontier from ordinal 0

TORN ROOTS

Root and witness checksums reject mixed spellings and partial installation. P is invisible to ordinary recovery.

TORN PAYLOAD

A trusted prefix plus CRC32C over the remaining bounded suffix prevents partial visibility.

THREAT BOUNDARY

CRC32C detects crash damage on trusted local disk; it is not tamper authentication.

The protocol never assumes that surviving writes form a prefix in submission order.

Append is the fast path; delete is ordered by phase

APPEND

write final tail once
publish longer root

no relocation or compaction

REWIND

publish shorter root
then `ftruncate` in place

restart repeats a lost truncate

DELETE

durable tombstone
then unlink + directory sync

no separate delete manifest

GROUP DELETE: FINISH THE RING WALK BEFORE CHANGING THE NAMESPACE

1. validate the complete closed ring
2. write every final M or T root
3. sync every participant inode
4. unlink T highest ordinal first
5. sync its parent before next unlink

M = retained materialized root
T = durable tombstone

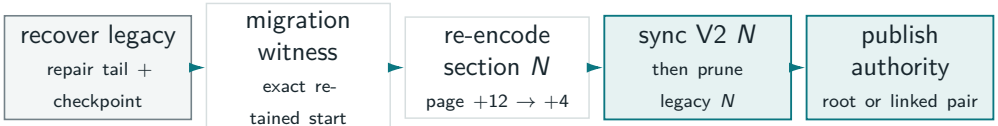
ordinal 0 stays present until last: the durable recovery anchor

After step 3, each M is independent. Descending, individually durable unlinks leave every remaining T in the forward prefix reachable from ordinal 0.

All final roots first; then a durable descending unlink frontier. Ordinal 0 replaces a delete manifest.

First contiguous V2 open: recover, replace, then switch authority

RESUMABLE, SECTION-AT-A-TIME CONVERSION



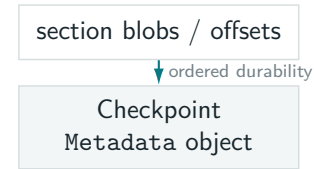
CRASH AUTHORITY

Ordinary V0/V1 opens/formats stay unchanged; generic `migrate_atomic` remains explicit. Peak migration space is V2 plus roughly one overlapping legacy section.

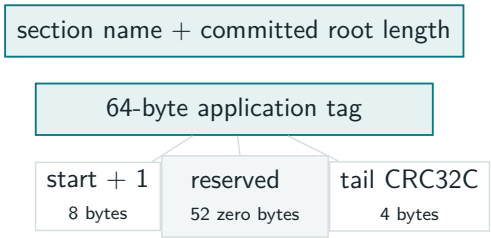
Init returns one V2-only journal; the mixed state exists only inside resumable migration.

Contiguous V2 removes its `Metadata` checkpoint

BEFORE



WITH ATOMIC V2 ROOTS

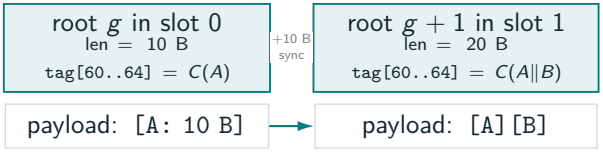


- Variable V2 publishes matching data + offsets roots in the **same UNO group**; neither can advance alone.
- One fixed section or variable pair holds the start; names + root lengths recover the remaining bounds.
- `MutableV2`: consuming mutations, `start_sync`, and `sync`—no legacy `commit` split.

Offsets remain as the variable index; the separate checkpoint object disappears.

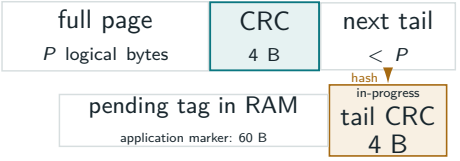
Small writes: the root checks the unfinished page

TWO 10-BYTE WRITES, EACH SYNCHRONIZED



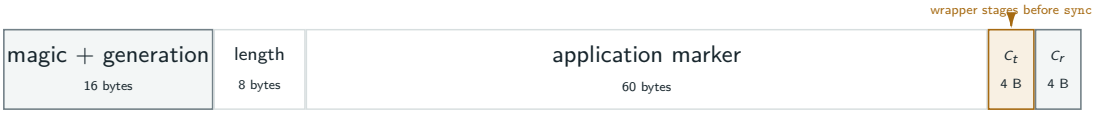
Torn root $g + 1$ or B $\Rightarrow$ use fallback g; the surviving suffix is unreachable.

WHEN THE PAGE BOUNDARY ARRIVES



- One footer per storage page, never per item.
- At most one unfooted tail; the selected root covers all of it.
- A completed page is immutable and cacheable.

92-BYTE ROOT HEADER INSIDE THE SELECTED 2 KIB ROOT/WITNESS SLOT



In progress: CRC in RAM **At sync:** root + local witness slot **Never at sync:** rewrite payload.

V2 caching: immutable payload pages, mutable RAM

WHAT REWRITES TODAY?

legacy partial physical tip

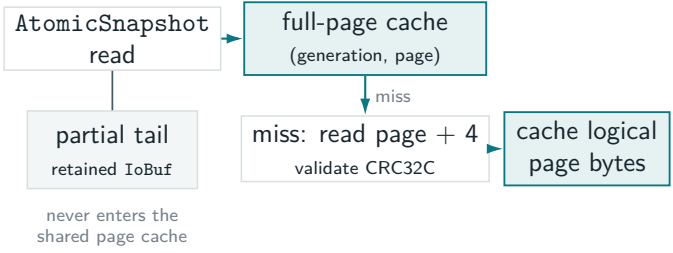
tail bytes len/CRC A len/CRC B

Append/rewind may rewrite that tip. The legacy tip's two length/CRC records retain the prior logical prefix; this is **not** arbitrary record overwrite.

CURRENT V2 IS DIFFERENT

`AtomicSnapshot` reads directly no `CacheRef`; cache-only reads miss

PLANNED V2 READ PATH



One cache core still coalesces faults and evicts RAM pages; loaders decode legacy page +12 or atomic page +4.

IDENTITY

One cache ID per opened blob generation; snapshots share it. A recreated name gets a new ID.

APPEND

A completed page is immutable and cacheable. The growing tail stays private and is written only by append.

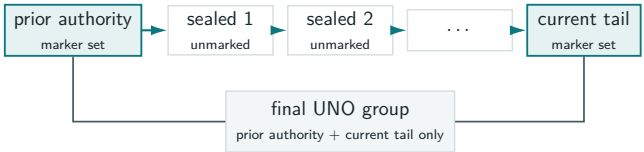
REWIND

Invalidate from the shortened page; publish the shorter root before physical offsets may be reused.

The cache may replace RAM entries; V2 never overwrites a completed payload page.

Rollover and reopen work stay bounded

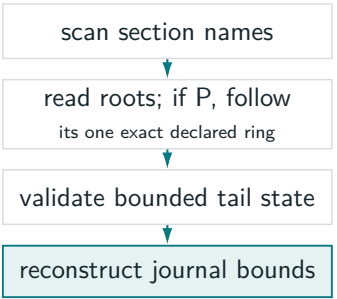
CROSSING MANY FIXED SECTIONS IN ONE EPOCH



Sealed intermediate sections may be durably staged while unreachable; their names fill the interval once the marker publishes.

fixed: **2 roots** variable: **4 roots**
prior/current × data/offsets, all in one group

OPEN / RESTART WORK



Fixed journals may read the final partial checked page of each section. Sealed variable sections are page-complete, so only active data/offset tails need equivalent validation.

Opening scans names and roots—never every frame, historical page, or complete blob.

Paired throughput: APFS 96.2%; ext4 98.0%

FINAL LINKED-WITNESS HOT PATH VS ORDINARY BLOBS

APFS ratios: `.986, .964, .945, .955, .962`
ext4 ratios: `1.005, .984, .958, .921, .980`

WORKLOAD

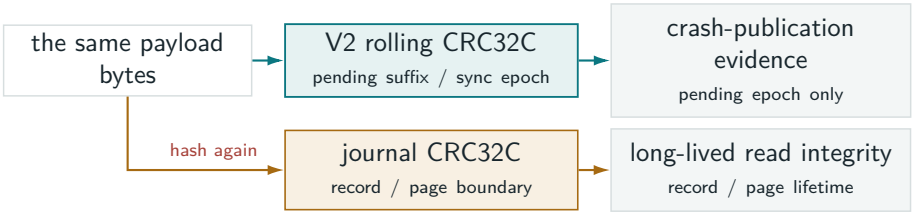
- 4 blobs per group
- sixteen contiguous 64 KiB appends per blob
- 4 MiB logical payload per group
- 250 groups per side; 5 paired seeds
- Tokio, 2 worker threads; sides interleaved

Environments
native macOS APFS; ext4 volume inside a Linux Docker VM

What remains non-free
local-link construction + locking; one durability barrier per participant

Payload is written once; the group has no coordinator file; variable data + offsets publish atomically.

Open question: can callers reuse V2's CRC work?



A POSSIBLE BRIDGE

- Accept caller-supplied spans; return composable `(offset, len, CRC32C)` receipts.
- Combine CRCs across write/preflush chunks without rereading bytes.
- Caller maps receipts through its index and persists them in group-published data—not the local witness or 64-byte tag.

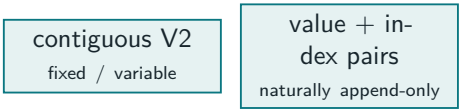
WHY THIS IS NOT AUTOMATIC

- A write may contain many records; one record may cross writes or preflush ranges.
- Byte offsets alone do not identify application items; variable journals need their offsets index.
- Root tags cannot hold a checksum map; storage-owned boundaries would recreate metadata.

Promising direction: share checksum computation, while callers retain ownership of logical boundaries.

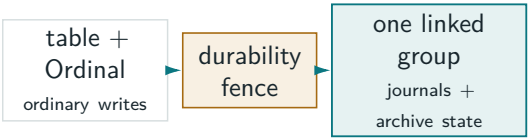
Open question: should every composite store adopt V2?

STOP AT APPEND-ONLY PRIMITIVES



- Existing stores retain Metadata and current durability ordering.
- Smallest compatibility and migration surface.
- Retains serialized checkpoint barriers in composite stores.

CONVERT COMPOSITE COMMIT BOUNDARIES



- Add opt-in V2 commit boundaries to Immutable Archive and peers.
- Fewer serial barriers; payload remains write-once.
- Requires a new format, migration, rollover, and fault tests; linked membership is not root-slot-sized.

"Convert all" does not mean making every file atomic.

Random-write indexes remain ordinary materializations; V2 owns the append-only journals and the authority that makes those materializations reachable.

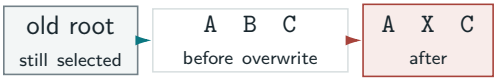
Keep V2 targeted—or make it the standard opt-in commit boundary for composite storage?

Atomic overwrite is a different storage engine

APPEND-ONLY: THE FALLBACK REMAINS INTACT

The old root still names `A B C`; selecting it discards `D` without restoring payload.

OVERWRITE: THE FALLBACK LOSES ITS BYTES



A CRC may reject torn `X`, but `B` is gone; the old root cannot restore it.

MAKING THAT OVERWRITE ROLLBACKABLE REQUIRES ANOTHER MECHANISM

COW / shadow pages

Write replacement elsewhere; publish a page map. Adds indirection and reclamation.

Undo / redo log

Persist old bytes or replay data first. Adds payload writes and another ordering protocol.

Volume layer

Version allocation below every store. Adds shared metadata, locking, and recovery work.

Under the fault model, any subset of unsynced overwrite bytes may survive; rollback therefore needs retained old bytes or a separate new copy.

CRC detects but cannot restore overwrites. Composite V2 keeps overwrite-heavy indexes ordinary behind atomic authority.